Rapport du POC : Recherche d'événements à Nantes

Synthèse technique et recommandations d'architecture : octobre 2026

Mathilde LE SOLLIEC

1. Objectif

Construire une première chaîne de préparation de données pour rechercher des événements culturels et publics à Nantes.

- Récupérer les événements depuis OpenAgenda via l'API OpenDataSoft.
- Nettoyer les données et harmoniser les informations utiles à la recherche.
- Transformer les événements en vecteurs et les enregistrer dans un index local.
- Proposer des recommandations personnalisées via un chatbot appuyé par le RAG.

2. Architecture du système

2.1 Architecture du RAG

- **1 : Collecte des données** via l'API OpenDataSoft.
- **2 : Prétraitement, nettoyage et standardisation.** Le nettoyage retire les espaces superflus et les données inutilisables, par exemple un événement sans titre. Le texte envoyé au modèle est ainsi plus lisible. La standardisation met les champs dans un format cohérent, par exemple en uniformisant les espaces et en nettoyant les champs texte. Elle reste volontairement légère pour ne pas modifier le sens des titres ou des descriptions.
- **3 : Vectorisation (modèle d'embedding).** Le modèle d'embedding transforme chaque document en vecteur. Deux textes au sens proche ont des vecteurs proches, comme deux points voisins sur une carte.

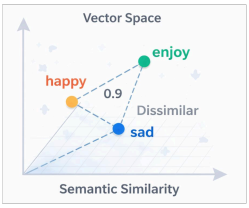


Figure 2 : Illustration de la vectorisation

- **4 : L'indexation (FAISS)** : FAISS prend tous ces vecteurs de chiffres et les organise dans une structure optimisée appelée **index vectoriel**, pour permettre des recherches par similarité.
- **5 : Interface utilisateur et requête.** La question est transformée en vecteur avec le même modèle que les événements. **Le vecteur de requête** est la représentation chiffrée de la question posée. Comme les événements et la question passent par le même modèle, leurs vecteurs sont comparables.
- **6 : Recherche dans FAISS.** FAISS est un ***moteur de recherche*** de vecteurs. Plusieurs méthodes permettent de comparer les vecteurs (clustering, comparaison exhaustive, compression des vecteurs...). Il renvoie les documents les plus proches dans l'espace vectoriel, qui sont potentiellement pertinents pour la question.
- **7 : Génération.** La question et les événements trouvés sont envoyés au LLM, qui génère une réponse en langage naturel.

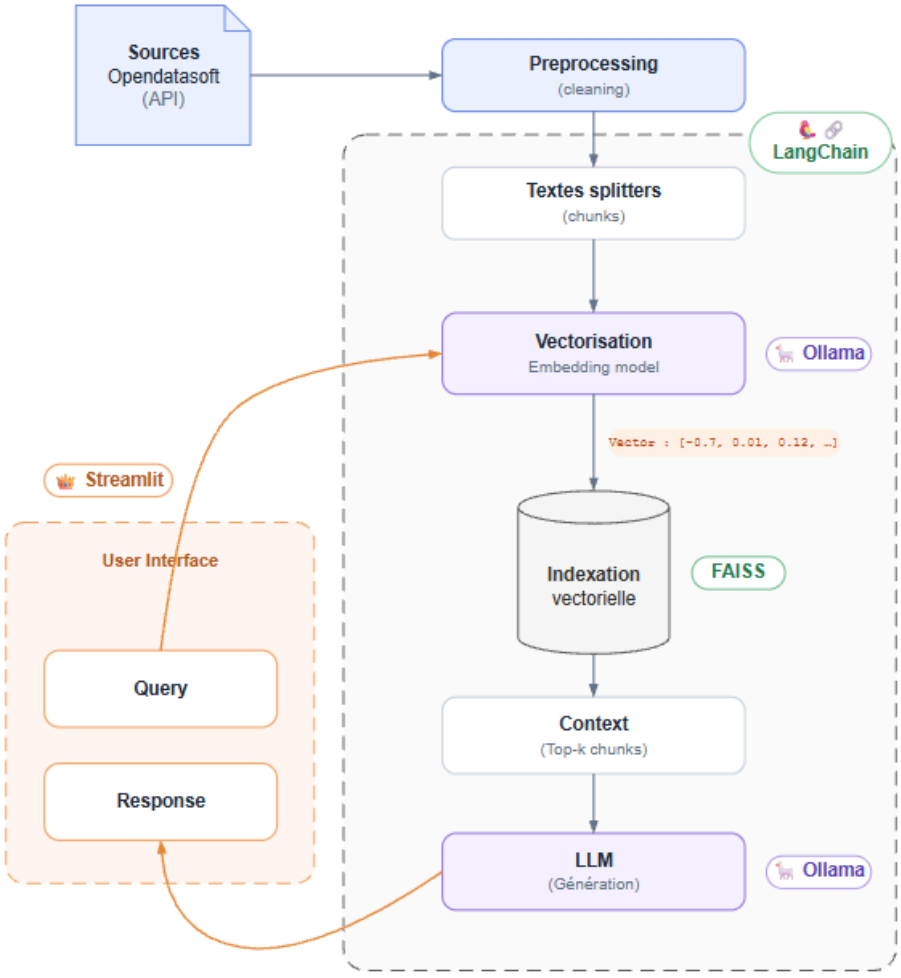


Figure 1 : Architecture du RAG

3. Choix techniques

3.1 Embeddings

Plusieurs approches permettent de transformer les événements en vecteurs. Elles se distinguent par leur mode d'exécution, le niveau de maîtrise des données, les coûts et les ressources nécessaires.

Tableau 1 : Comparaison des solutions pour les embeddings et la génération

Type de solution	Rôle et exemples	Sécurité et données	Installation	Coût (10k docs)
Local / CPU	Vectorisation (ex. Sentence Transformers)	Maximale (100 % isolée), tourne dans le code	Package local	0 € / token
Moteur local + LLM (GPU)	Génération et embeddings avec de petits modèles (ex. Ollama, LLaMA 8B)	Maximale (100 % isolée), aucune donnée externe	Binaire local	0 € / token
API Cloud SaaS (UE/US)	Génération gérée à distance (Mistral, GPT-4o, Gemini)	Moyenne à forte (Mistral UE RGPD / US Cloud Act)	Clé API Cloud	estimé à 10 € / mois (au token)
Auto-hébergement Cloud	Modèles géants (LLaMA 70B, Mixtral 8×22B)	Excellente (infrastructure sous contrôle direct)	Docker / vLLM	estimé 30 € / mois (serveur GPU 2h)

✅ **Choix retenu : Ollama en local.** L'embedding et la génération tournent sur la machine, sans appel à une API externe et sans coût au token. Le POC a d'abord été écrit avec l'API Mistral, puis basculé sur Ollama : `ollama pull qwen3:4b`.

3.2 Stockage vectoriel

Tableau 2 : Comparaison des solutions de stockage vectoriel

Solution	Type	Hébergement	Passage à l'échelle	Coût	Données locales
FAISS	Bibliothèque Python de recherche vectorielle	Local (dans l'app Python)	Limité par la machine	Gratuit (infra à financer)	Oui
Weaviate	Base de données	Local ou Cloud	Adapté à la montée en	Dépend du déploiement	Oui (si auto-hébergé)

Solution	Type	Hébergement	Passage à l'échelle	Coût	Données locales
	vectorielle avec filtres		charge		
Milvus	Base vectorielle distribuée multi-machines	Local ou Cloud	Conçu pour de très grands volumes	Dépend du déploiement	Oui (si auto-hébergé)
Pinecone	Base vectorielle managée (Cloud)	Cloud	Adapté à la montée en charge	Dépend de l'offre Cloud	Non

Choix retenu : FAISS. L'index est conservé localement dans les fichiers `index.faiss` et `index.pkl`. La bibliothèque permet des recherches de similarité sans service de base vectorielle externe.

FAISS convient à un POC ou à un volume moyen. Il charge l'index en RAM pour être rapide. À grande échelle, une **base vectorielle** dédiée (Qdrant, Milvus, Weaviate, pgvector) peut être envisagée. Si les données changent, l'index doit être mis à jour ; une reconstruction complète peut être nécessaire selon la méthode utilisée.

3.3 Algorithme d'indexation FAISS

Tableau 3 : Comparaison des algorithmes d'indexation FAISS

Algorithme	Fonctionnement	Avantages	Inconvénients
IndexFlatL2	Compare le vecteur recherché à tous les vecteurs enregistrés, en calculant la distance euclidienne.	Très précis, simple à utiliser.	Recherche plus lente quand le volume augmente.
IndexFlatIP	Compare les vecteurs avec le produit scalaire (Inner Product).	Très précis, adapté à la similarité cosinus avec des vecteurs normalisés.	Compare tous les vecteurs.
IndexIVFFlat	Divise les vecteurs en groupes (clusters) et ne recherche que dans certains groupes.	Recherche plus rapide sur de gros volumes.	Peut manquer le meilleur résultat si le bon groupe n'est pas exploré.
IndexHNSW	Organise les vecteurs dans un graphe pour naviguer rapidement vers les voisins les plus proches.	Très rapide, bonne précision.	Consomme davantage de mémoire.

✅ **Choix retenu : IndexFlatIP.** Pour environ 10 000 événements, la recherche exhaustive reste raisonnable. Elle compare les vecteurs par produit scalaire, ce qui convient à la similarité cosinus lorsque les vecteurs sont normalisés.

Concrètement :

- **IndexFlat : comparaison exhaustive.** FAISS compare la question aux 10 000 événements, un par un, calcule leur proximité et retourne les plus proches. Aucun meilleur résultat n'est manqué, mais tout est comparé.
- **IndexIVF : recherche par groupes.** Les vecteurs des événements sont répartis en clusters selon leur proximité. Une question sur les concerts n'explore que les groupes les plus proches : moins de comparaisons, donc une recherche plus rapide, mais des événements pertinents peuvent être manqués.
- **IndexHNSW : recherche par graphe.** Chaque vecteur est relié à ses voisins proches et la recherche progresse de proche en proche. Très rapide, mais plus gourmand en mémoire et sans garantie du meilleur résultat absolu.

Autres choix :

- **CSV comme format intermédiaire** : simple à inspecter et à réutiliser entre les étapes. Le fichier brut est conservé séparément du fichier nettoyé.
- **Pandas pour les transformations** : validation, normalisation, filtrage et déduplication de lignes tabulaires.
- **Un événement par document** : les informations d'un même événement restent ensemble, et les métadonnées permettent de rattacher un résultat à sa source. Aucun découpage par caractères n'est appliqué.
- **LangChain** : chaque outil (Ollama, Mistral, OpenAI, FAISS...) a sa propre façon de s’utiliser, ce qui peut nécessiter beaucoup de code de liaison. LangChain est un framework qui facilite les liaisons entre les outils.

3.4 Logigramme du RAG

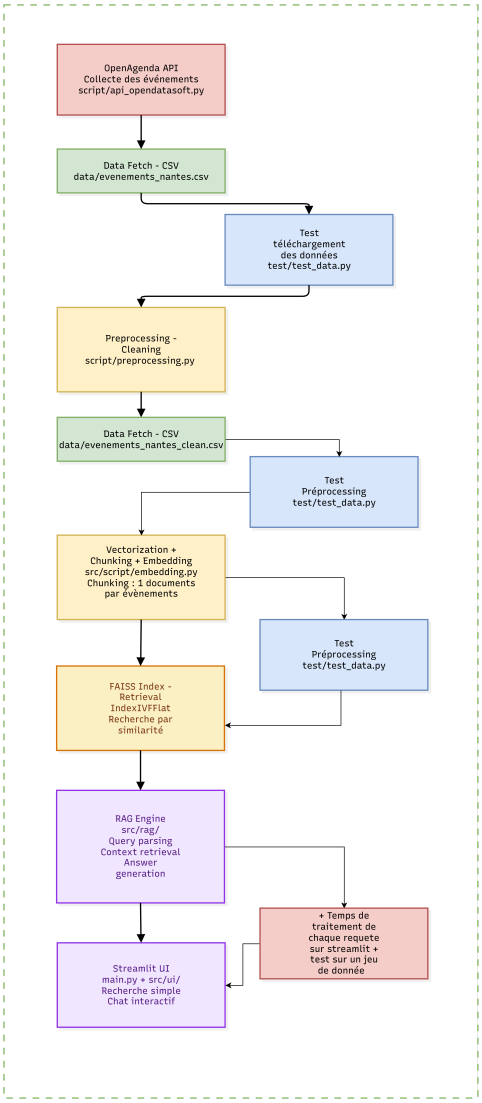


Figure 3 : Logigramme du RAG

4. Résultats obtenus

Tableau 4 : Résultats du POC

Étape	Résultat
Données	10 000 événements reçus, 9 841 conservés, 159 écartés (hors ville ou trop anciens). Zéro doublon, zéro date invalide.
Index vectoriel	`index.faiss` et `index.pkl` générés dans `data/faiss_index/` : 9 841 vecteurs, dimension 1 024. Index daté du 23 octobre 2026.
Tests	Tests unitaires de création de documents et d'index FAISS simulé réussis. Dans une exécution de `tests/test_embeddings.py` : 2 tests réussis, 2 tests d'intégration Ollama ignorés. Une autre exécution locale, avec les tests d'intégration, rapporte 12 tests réussis sur l'ensemble du pipeline.

5. Méthode pour garantir la qualité des données

Tests et intégration continue

La qualité du pipeline est vérifiée par **12 tests automatisés (pytest)**, répartis sur trois niveaux :

- **Tests unitaires du prétraitement** (`tests/test_preprocessing.py`) :
  - **Présence des colonnes obligatoires** dans la structure des données.
  - **Absence de valeurs nulles ou vides dans les champs obligatoires**, notamment `title_fr` (titre de l'événement).
  - **Validité des dates et cohérence de leur ordre chronologique** (`lastdate_end >= firstdate_begin`).
  - **Vérification des filtres géographiques attendus et suppression des doublons**.
- **Tests de qualité des données** (`tests/test_data.py`). Ils contrôlent le jeu de données réel : présence des colonnes obligatoires, titres renseignés, dates lisibles et cohérentes (début avant fin), identifiants uniques, événements situés à Nantes et dans la période retenue.
- **Tests de la vectorisation** (`tests/test_embeddings.py`). Ils vérifient qu'un document est créé par événement avec ses métadonnées, et que l'index FAISS est bien construit et sauvegardé. Ces tests utilisent un **modèle d'embedding simulé**, ce qui les rend rapides et indépendants d'Ollama. Deux **tests d'intégration**, activés avec `RUN_OLLAMA_INTEGRATION=1`, appellent le vrai modèle local et vérifient qu'il renvoie des vecteurs de dimension constante.

Résultat : 12 tests réussis sur 12, en environ 9 secondes avec les tests d'intégration.

Les tests sont exécutés automatiquement par **GitHub Actions** à chaque *push* et à chaque *pull request* sur la branche `main`. Un test en échec signale une régression ; le blocage de la fusion dépend des règles de protection de la branche. Les tests d'intégration, qui ont besoin d'Ollama, ne tournent qu'en local.

6. Évaluation de la qualité des réponses du chatbot

Indicateurs à suivre pour ce RAG :

- **Faithfulness** : fidélité de la réponse aux sources, afin de limiter les hallucinations.
- **Pertinence** : la réponse répond-elle vraiment à la question ?
- **Recall@k** : quelle proportion des événements attendus est présente parmi les k documents récupérés ?

Un premier jeu de cinq questions de référence et plusieurs indicateurs sont présentés ci-dessous. Ce jeu reste limité : il est à élargir pour évaluer la qualité métier et la fidélité aux sources, puis à exécuter régulièrement à chaque modification.

Étape 1 : Préparer le jeu de référence

Les **5 questions** ont été écrites puis annotées à la main à partir des vrais événements du CSV. Chacune couvre un cas différent, y compris des cas difficiles :

Tableau 5 : Questions du jeu de référence

Id	Question	Ce qu'elle teste
q01	« Que faire ce week-end avec un enfant de 4 ans ? »	Période + âge du public
q02	« Comment participer à la Nuit sans fin consacrée à Steve Reich au Conservatoire ? »	Détails pratiques d'un événement
q03	« Un spectacle d'improvisation ce soir ? »	Période courte + thème précis
q04	« Y a-t-il un concert de Noël le week-end prochain ? »	Un seul bon événement parmi d'autres concerts
q05	« Quels événements sont prévus à Lyon ce week-end ? »	Refus : seule Nantes est couverte

Voici, en simplifié, ce que contient le corrigé de la question q04 :

```
{
  "id": "q04",
  "question": "Y a-t-il un concert de Noël le week-end prochain ?",
  "date_reference": "2026-11-13T10:00:00+01:00",
  "periode_attendue": "week-end prochain",
  "uids_attendus": ["34650300"],
  "reponse_attendue": "Concert de Noël de l'Ensemble vocal de Nantes à
l'église Notre-Dame de Bon Port, dimanche 22/11/2026 de 17h00 à 18h3
0.",
  "elements_cles": ["Ensemble vocal", "Bon Port", "22/11", "17h"],
  "doit_refuser": false
}
```

La commande `python -m script.evaluation` prend les questions une par une. Pour chacune :

1. le chatbot reçoit la question **avec la date simulée** ;
2. il fait sa recherche dans FAISS : on garde la liste des 5 événements trouvés ;
3. il rédige sa réponse avec le LLM : on garde le texte ;
4. le script compare les deux au corrigé et calcule les notes.

Une erreur peut venir de la recherche (les bons événements n'ont pas été trouvés) ou de la génération (ils ont été trouvés, mais mal utilisés par le LLM).

Tableau 7 : Notes de la recherche : a-t-on trouvé les bons événements ?

Indicateur	Question posée	Exemple tiré des résultats
Recall@5 (rappel)	Parmi les événements attendus, combien sont dans les 5 trouvés ?	q03 : 4 spectacles d'impro attendus, 1 seul trouvé → 1 / 4 = **0,25**.
Rang réciproque et MRR	Le premier bon événement arrive-t-il en tête de liste ?	Rang réciproque : en 1re position → 1 ; en 2e → 0,5 ; en 4e → 0,25 ; absent → 0. Le MRR est la moyenne de ces scores sur plusieurs questions.
Precision@5 (précision)	Parmi les 5 trouvés, combien sont attendus ?	Peu parlant ici : quand un seul événement est attendu, elle ne peut pas dépasser 1 / 5.

Tableau 8 : Notes de la réponse : le chatbot a-t-il bien répondu ?

Indicateur	Question posée	Exemple tiré des résultats
Couverture des éléments clés	Les informations indispensables sont-elles dans la réponse ?	q04 : sur « Ensemble vocal », « Bon Port », « 22/11 » et « 17h », seul « 22/11 » apparaît → 1 / 4 = **0,25**. Le chatbot a inventé un autre concert.
Similarité sémantique	La réponse a-t-elle **le même sens** que la référence ?	Les deux textes sont transformés en vecteurs, puis comparés (de 0 à 1). q01 : 0,86, sens très proche. q05 : 0,53, sens éloigné.
F1 lexical	La réponse utilise-t-elle **les mêmes mots** que la référence ?	Mesure complémentaire, sévère dès que le chatbot reformule.
Refus correct	Le chatbot dit-il « je n'ai rien trouvé » quand c'est la bonne réponse, et seulement dans ce cas ?	q05 : il propose des événements nantais pour Lyon → **échec**.

7. Limites du POC et recommandations pour la suite

- Récupérer les données supplémentaires sans avoir à relancer le modèle d'embedding pour l'ensemble des données.
- Rendre le POC scalable en adaptant les outils utilisés.
- Ajuster le modèle pour maximiser la qualité des réponses.
- Évaluer les réponses avec RAGAS sur des questions de référence.